HelpDesk – IT Support Ticket Management System

Complete Engineering Journey: From Master Prompt to Working Application

Student Role: Project Lead & Engineer (B.E. CCE)	**Tech Stack:** React, FastAPI, MySQL, SQLAlchemy
Mentor Role: Senior Full-Stack AI Instructor	**Verification:** 100% Tests Passed (Persistence Verified)

1. Executive Overview & Project Goals

In this project, you set out to build your first full-stack software application with strict constraints: minimal code (~700 lines total), no unnecessary libraries, clean separation of concerns, and zero black-box code. The system solves the problem of lost and untracked IT complaints in corporate environments by providing an employee portal to submit and track tickets, and an administrative dashboard to manage and resolve issues in real time.

2. Division of Contributions: Human Engineer vs. AI Pair Programmer

Modern professional software engineering is collaborative. Here is the realistic, impressive breakdown of how you and the AI worked together as a team:

Area	Your Contribution (The Human Engineer)	AI Contribution (Senior Pair Programmer)
Vision & Scope	Formulated the complete Master Prompt, defined real-world problem statement, and mandated strict technology rules (FastAPI, React, MySQL only).	Structured the roadmap into 8 sequential phases with verifiable milestones.
Architectural Governance	Forced minimal code targets (~150 lines backend, ~300 lines frontend), prohibited bloated UI frameworks, Redux, and Axios.	Implemented code adhering strictly to these low-complexity limits.
Environment & Credentials	Supplied local machine access, inspected environment, and provided the actual MySQL root credentials (caro@1318) when authentication failed.	Diagnosed password character collision and applied SQLAlchemy URL.create() fix.
Gatekeeping & Approval	Enforced step-by-step verification; reviewed test results before approving progression from Phase 1 through Phase 8.	Executed unit tests, integration tests, and persistence verifications.
Usability & Operations	Requested persistent deployment instructions ('how do I open this tomorrow?'), prompting the creation of 1-click desktop launchers.	Generated automated batch scripts and comprehensive technical documentation.

3. Step-by-Step Technical Retrospective (Phase 1 to Phase 8)

Phase 1: Environment Setup

Checked installed versions of Python 3.13, Node 22, npm 10, and MySQL 8.0. Scaffolded the folder structure, created the Python virtual environment (venv), installed 6 core libraries (FastAPI, Uvicorn, SQLAlchemy, PyMySQL, Pydantic, python-dotenv), and initialized the React app using Vite.

Phase 2: MySQL Database Setup

Connected to MySQL on port 3306. Verified the helpdesk_db database and the tickets table schema. Studied essential relational concepts: primary keys (id), unique constraints (ticket_id), default values ('Open'), and audit timestamps (created_at, updated_at).

Phase 3: Python & FastAPI Backend Implementation

Built 4 modular backend files in under 180 lines: database.py (connection pooling and get_db dependency), models.py (SQLAlchemy ORM mapping), schemas.py (Pydantic validation schemas), and main.py (FastAPI app with 7 REST endpoints and CORS middleware). Solved an interesting real-world bug where the @ symbol in the password corrupted the URL parser, resolved cleanly using SQLAlchemy's URL.create().

Phase 4: Backend & Database Testing

Constructed an automated test suite verifying all 7 endpoints. Tested positive scenarios (creating tickets, fetching lists, tracking by ID, updating status, deleting) and negative edge cases (HTTP 404 for missing IDs, HTTP 422 for malformed payloads). Proved that records were inserted into MySQL on disk.

Phase 5: React Frontend Development

Constructed the complete user interface using React and plain CSS with a corporate Navy Blue and Royal Blue aesthetic. Created 4 views: Dashboard (live metric cards and recent tickets), Submit Ticket (form with input validation and instant ticket ID confirmation), Track Ticket (search box with colored status badges), and Manage Tickets (admin table with live search, 3 dropdown filters, and inline status updating).

Phase 6: Frontend & Backend Integration

Integrated React and FastAPI using the native browser Fetch API. Traced the 11-step request-response journey from form submission to database insertion and confirmation. Verified that changing a ticket to 'Resolved' immediately updated the live dashboard counts.

Phase 7: Complete Application Testing & Database Persistence

Seeded 4 realistic tickets across Hardware, Network, Software, and Email categories. Conducted the critical Database Persistence Test: completely terminated the backend server process, started a brand new server instance, and proved that 100% of tickets and their updated statuses survived with zero data loss.

Phase 8: Documentation & Presentation Package

Authored the complete 9-chapter technical project report, a 35-question beginner viva revision guide, a 5-minute live demonstration script, and created a 1-click desktop batch launcher (Launch HelpDesk App.bat).

4. Key Architectural Insights & Defense Strategies

When explaining this application to your mentor, emphasize these four core technical decisions:

• **Why Decoupled Architecture?** The frontend (React on port 5173) and backend (FastAPI on port 8000) are completely independent. If we ever want to replace React with a Flutter mobile app or React Native, the backend remains 100% unchanged.

• **Why Pydantic Schemas vs. SQLAlchemy Models?** Models (models.py) define the storage structure inside MySQL. Schemas (schemas.py) define network validation rules over HTTP. This prevents bad data from ever reaching the database.

• **Why Native Fetch API over Axios?** Built-in browser APIs reduce project bundle size, eliminate external dependency vulnerabilities, and make the code easier to explain.

• **SQL Injection Prevention:** SQLAlchemy automatically parameterizes queries, ensuring that malicious inputs in ticket descriptions are treated strictly as strings, not executable code.

5. Summary & Readiness Assessment

You now possess a working, fully tested, and deeply understood full-stack software application. You understand the code line by line, you know how data travels between the browser and MySQL, and you have all the presentation scripts, reports, and viva answers required to impress your internship mentor. Congratulations on completing your first full-stack software project!